MLFP mlfp06 — Speaker Notes

- Module 6: Machine Learning with Language Models and Agentic Workflows — Speaker Notes
  - Slide 1: Title — Module 6: Machine Learning with Language Models and Agentic Workflows
  - Slide 2: What You Will Learn
  - Slide 3: Your Journey — 8 Lessons
  - Slide 4: Kailash Engines You Will Meet
  - Slide 5: Where We Are
  - Slide 6: Lesson 6.1 — LLM Fundamentals, Prompt Engineering & Structured Output
  - Slide 7: How LLMs Learn — Pre-training
  - Slide 8: Scaling Laws — Parameters, Data, Compute
  - Slide 9: From Prediction to Helpfulness — RLHF
  - Slide 10: Prompt Engineering — Zero-shot & Few-shot
  - Slide 11: Chain-of-Thought Prompting
  - Slide 12: Self-Consistency & Structured Prompting
  - Slide 13: Prompt Engineering — The Complete Toolkit
  - Slide 14: Kaizen — Delegate & Signature
  - Slide 15: Inference Optimisation (Brief)
  - Slide 16: Exercise 6.1 — Prompt Engineering Showdown
  - Slide 17: Lesson 6.2 — LLM Fine-tuning, LoRA, Adapters & the Technique Landscape
  - Slide 18: LoRA — Low-Rank Adaptation
  - Slide 19: LoRA — From-Scratch Implementation
  - Slide 20: Adapter Layers — Bottleneck Modules
  - Slide 21: LoRA vs Adapters — Head-to-Head
  - Slide 22: The Fine-tuning Landscape — 10 Techniques
  - Slide 23: Model Merging — Combining Fine-tuned Models
  - Slide 24: Quantisation — Running Large Models on Small Hardware
  - Slide 25: kailash-align — Fine-tuning with AlignmentPipeline
  - Slide 26: Exercise 6.2 — From-Scratch Fine-tuning
  - Slide 27: Lesson 6.3 — Preference Alignment: DPO & GRPO
  - Slide 28: DPO — Direct Preference Optimization
  - Slide 29: DPO — Training Loop
  - Slide 30: GRPO — Group Relative Policy Optimization
  - Slide 31: LLM-as-Judge Evaluation
  - Slide 32: Evaluation Benchmarks
  - Slide 33: kailash-align — DPO Training
  - Slide 34: Exercise 6.3 — Preference Alignment
  - Slide 35: Lesson 6.4 — RAG Systems
  - Slide 36: RAG — Retrieval-Augmented Generation
  - Slide 37: Chunking Strategies
  - Slide 38: Retrieval — Dense, Sparse, and Hybrid
  - Slide 39: Re-ranking & HyDE
  - Slide 40: RAGAS — RAG Evaluation Framework
  - Slide 41: Exercise 6.4 — Build a RAG System
  - Slide 42: Lesson 6.5 — AI Agents: ReAct, Tool Use & Function Calling
  - Slide 43: What Is an AI Agent?
  - Slide 44: ReAct — Reasoning + Acting
  - Slide 45: Function Calling — Structured Tool Use
  - Slide 46: Agent Design — The Hiring Framework
  - Slide 47: Kaizen — Building Agents
  - Slide 48: Cost Budgets — Preventing Runaway Spending
  - Slide 49: Exercise 6.5 — Build a Data Analysis Agent

- Slide 50: Lesson 6.6 — Multi-Agent Orchestration & MCP
- Slide 51: Multi-Agent Patterns
- Slide 52: Multi-Agent — Supervisor-Worker in Kaizen
- Slide 53: Agent Memory
- Slide 54: MCP — Model Context Protocol
- Slide 55: A2A — Agent-to-Agent Communication
- Slide 56: Exercise 6.6 — Multi-Agent ML Pipeline
- Slide 57: Lesson 6.7 — AI Governance Engineering
- Slide 58: PACT — D/T/R Addressing
- Slide 59: Operating Envelopes
- Slide 60: Budget Cascading
- Slide 61: PactGovernedAgent — Governance Built In
- Slide 62: Governance Testing — Proving Safety
- Slide 63: Audit Trails & Clearance Levels
- Slide 64: Exercise 6.7 — Governed Multi-Agent System
- Slide 65: Lesson 6.8 — Capstone: Full Production Platform
- Slide 66: Nexus — One Codebase, Three Interfaces
- Slide 67: Authentication & Authorisation
- Slide 68: Full Platform Integration
- Slide 69: Production Monitoring — DriftMonitor
- Slide 70: Debugging Agent Reasoning Chains
- Slide 71: Testing Agentic Systems
- Slide 72: Inference Optimisation — Production Serving
- Slide 73: Multimodal LLMs — Brief Awareness
- Slide 74: Exercise 6.8 — Capstone: Deploy a Governed AI Platform
- Slide 75: Key Formula Recap — Attention Mechanism
- Slide 76: M6 Formula Summary
- Slide 77: The Complete Kailash Stack
- Slide 78: The M6 Journey
- Slide 79: What Makes This Module Different
- Slide 80: Decision Framework — When to Use What
- Slide 81: Common Mistakes to Avoid
- Slide 82: M6 in the MLFP Curriculum
- Slide 83: The ML Spectrum — Where LLMs Fit
- Slide 84: Self-Assessment — Can You…
- Slide 85: End-of-Module Assessment
- Slide 86: Resources & References
- Slide 87: Key Takeaways
- Slide 88: For the Instructor — Pacing Notes
- Slide 89: Discussion Questions
- Slide 90: Module 6 Complete
- Instructor Notes
  - Pacing Summary
  - Audience Calibration
  - Critical Messages (Repeat Often)
  - Final Note

Module 6: Machine Learning with Language Models and Agentic Workflows — Speaker Notes

Total time: ~180 minutes (3 hours) Audience: working professionals; instructors must scaffold for both novices and LLM practitioners.

This is the final module of MLFP. By the end of today, students will have built, fine-tuned, aligned, grounded, governed, and deployed a production AI system. Pace the room accordingly: treat the capstone as the destination, and treat every earlier lesson as a tool they will pick up in the final hour.

Slide 1: Title — Module 6: Machine Learning with Language Models and Agentic Workflows

Time: ~2 min **Talking points**:

- Welcome to the final module of MLFP. Read the provocation aloud: “The best LLM application is not a chatbot. It is an engineer that reasons, acts, and governs itself.”
- Let it sit. This frames the entire module: we are not here to prompt a chatbot, we are here to build systems.
- Emphasise the pivot: every previous module was about models that take input and produce output. Today we build systems that think, plan, and act autonomously.
- Ask for a show of hands: “Who has used ChatGPT or Claude?” Most will raise their hands. “Today you learn to build one.”
- “If beginners look confused”: “If you have only ever typed into a chat box, today you see the engineering underneath. We will go slowly through the fundamentals before we build anything.”
- “If experts look bored”: “We implement LoRA from scratch, derive the DPO loss term by term, and build a governed multi-agent system with PACT. The theory track will earn its keep.” **Transition**: “Let me show you exactly what you will be able to do by the end of today.”

Slide 2: What You Will Learn

Time: ~2 min **Talking points**:

- Walk through the eight outcomes on the left column. These are concrete capabilities, not abstract concepts.
- Introduce the three depth layers: FOUNDATIONS (green, everyone), THEORY (blue, stretch), ADVANCED (purple, experts).
- “A product manager and a research engineer sit in the same classroom. Both leave having learned something new. This module is designed for both.”
- “If beginners look confused”: “Follow the green markers. You will come out able to build RAG systems and governed agents. You can skip the derivations — they are labelled Theory.”
- “If experts look bored”: “The DPO derivation slide and the GRPO advantage formula are purple. That is where you will find new depth.” **Transition**: “Here is the roadmap for our 8 lessons today.”

Slide 3: Your Journey — 8 Lessons

Time: ~1 min **Talking points**:

- Walk through the table once. Do not dwell on each cell.
- Emphasise the arc: understand LLMs (6.1), customise them (6.2), align them (6.3), ground them (6.4), give them agency (6.5), coordinate them (6.6), govern them (6.7), ship them (6.8).
- “Each lesson produces runnable code. By the end, you have a deployed AI platform.”
- “If beginners look confused”: “The first four lessons teach you what LLMs do. The last four teach you how to ship them safely.”
- “If experts look bored”: “6.2 and 6.3 are the densest. Focus your attention there.” **Transition**: “To build all of that, you will use four Kailash packages.”

Slide 4: Kailash Engines You Will Meet

Time: ~2 min **Talking points**:

- Introduce the four packages briefly. Kaizen: agent framework and LLM calls. kailash-align: fine-tuning and preference alignment. kailash-pact: governance and access control. kailash-nexus: multi-channel deployment.
- Mention kailash-mcp briefly — it is the MCP server library we will use in 6.6.
- "Every exercise uses at least one of these. By the end of 6.8, you will have used all of them in a single deployed system."
- "If beginners look confused": "Think of these as specialised tools. Kaizen is the one you will touch most often today."
- "If experts look bored": "AdapterRegistry, OperatingEnvelope, and PactGovernedAgent are the three classes most experts have not seen before." **Transition**: "Before we dive in, let us locate ourselves in the MLFP journey."

Slide 5: Where We Are

Time: ~2 min **Talking points**:

- Walk through the table: M1 data, M2 statistics, M3 supervised ML, M4 unsupervised and NLP, M5 deep learning and transformers.
- "Today we take the transformer architecture from M5 and give it the ability to reason, act, learn from feedback, and govern itself. Everything you built before becomes a tool an agent can wield."
- "If beginners look confused": "Think of M5 as building the engine. M6 is building the car around it."
- "If experts look bored": "The interesting bit is how LoRA ties back to M4's SVD. We will reach that in 6.2." **Transition**: "Let us begin with the most immediately practical skill: getting good output from an LLM."

[PAUSE FOR QUESTIONS — 1 min]

Slide 6: Lesson 6.1 — LLM Fundamentals, Prompt Engineering & Structured Output

Time: ~1 min **Talking points**:

- State the four learning objectives on screen. Read them aloud.
- "This lesson covers the most immediately practical LLM skill: getting good output from a model. We start with how they work, then focus on prompt engineering, then structured output with Kaizen Delegate."
- Prerequisite check: "Everyone completed M5? The transformer architecture is assumed knowledge. If anything feels unfamiliar, flag it at the break." **Transition**: "Let us start with how LLMs learn in the first place."

Slide 7: How LLMs Learn — Pre-training

Time: ~3 min **Talking points**:

- Explain the GPT-style objective: “Given a sequence of tokens, predict the next one. That is literally all it does during pre-training. Yet from this simple objective, it learns grammar, facts, and reasoning.”
- Walk through the loss function slowly. Point at the summation: “We are minimising the negative log probability of the correct next token across the whole corpus.”
- Contrast with BERT: masked language modelling is bidirectional — better for classification, worse for generation.
- “Pre-training does not teach the model to follow instructions. It teaches it to complete text. Alignment is what makes it helpful — we get to that in 6.3.”
- “If beginners look confused”: “Imagine being given a sentence with the last word missing, over and over, trillions of times. You would get very good at language. That is what the model is doing.”
- “If experts look bored”: “The cross-entropy form is standard; the interesting bit is the data scale. Chinchilla argues the field over-prioritised parameters at the expense of data — we see that next.” **Transition**: “How big should these models be, and how much data do they need?”

Slide 8: Scaling Laws — Parameters, Data, Compute

Time: ~3 min **Talking points**:

- Core message: “Bigger models plus more data equals better performance, and the relationship is predictable.”
- Explain Kaplan’s scaling laws: loss decreases as a power function of model size, dataset size, and compute.
- Chinchilla (2022) correction: “Many early models were undertrained — too many parameters, not enough data. Chinchilla showed you should scale parameters and tokens roughly equally.”
- Walk through the notable models table briefly. Do not dwell. Point to the diversity: Mixture of Experts, constitutional training, open weights, sliding window attention, knowledge distillation, high-quality data.
- “If beginners look confused”: “You do not need to memorise these. The point is the landscape moves fast. The engineering patterns we teach today apply to every model in this table.”
- “If experts look bored”: “Ask yourselves: is Phi-3 at 3.8B the most important data point here? Data quality beats parameter count, and the whole curve bends with it.” **Transition**: “Pre-training gives us a fluent model. But fluency is not helpfulness. How do we make it actually useful?”

Slide 9: From Prediction to Helpfulness — RLHF

Time: ~3 min **Talking points**:

- Walk through the flow: pre-trained LLM, supervised fine-tuning (SFT), reward model training, PPO against the reward model, aligned LLM.
- “Pre-training optimises prediction. SFT teaches format — how to follow instructions. RLHF teaches quality — how to be helpful, harmless, and honest.”
- Flag the complexity: two models in memory, PPO instability, reward model drift. “This is why only big labs did RLHF for years.”
- Connect to M5: “Remember PPO from M5? It is used here to optimise the LLM against a reward model. In Lesson 6.3 we will see DPO, which skips the reward model entirely.”
- “If beginners look confused”: “Think of RLHF as teaching the model what ‘good’ looks like by showing it examples of human preferences.”
- “If experts look bored”: “The interesting question is whether the reward model is learning the right signal. DPO argues you can skip that learning step and go directly from preferences to policy.” **Transition**: “Before we get to alignment, let us master the cheapest and most powerful LLM skill: prompt engineering.”

Slide 10: Prompt Engineering — Zero-shot & Few-shot

Time: ~3 min **Talking points**:

- Walk through zero-shot: "Task description only, no examples. Works well for simple, well-defined tasks."
- Walk through few-shot: "Provide a couple of examples to set the pattern. Works much better for structured or domain-specific tasks."
- "Few-shot is in-context learning: the model infers the task from examples without any weight updates. No training, just examples in the prompt."
- If you can do a live demo, run both prompts against the same input. Show the difference.
- "If beginners look confused": "Zero-shot is asking someone to do a task with just instructions. Few-shot is showing them examples first. Most of us perform better with examples — so do LLMs."
- "If experts look bored": "The in-context learning phenomenon is still not fully understood theoretically. There are several competing hypotheses in the literature." **Transition**: "For reasoning tasks, there is a single phrase that changes everything."

Slide 11: Chain-of-Thought Prompting

Time: ~3 min **Talking points**:

- Walk through standard CoT: few-shot with reasoning included in each example.
- Walk through zero-shot CoT: just append "Let's think step by step" — no examples needed.
- Show the Kojima et al. (2022) result: GSM8K accuracy jumps from 17.7% to 78.7% with one phrase. "This is not a small improvement. This is a 4x gain from seven words."
- "CoT forces the model to show its work, which prevents it from jumping to wrong conclusions."
- "If beginners look confused": "It is like asking a student to show their working on a maths exam. They get partial credit — and they also get the right answer more often."
- "If experts look bored": "The interesting theoretical question: is CoT activating a latent reasoning capability, or is it just spreading computation across more tokens? The answer seems to be both." **Transition**: "CoT is good. Self-consistency on top of CoT is better."

Slide 12: Self-Consistency & Structured Prompting

Time: ~3 min **Talking points**:

- Self-consistency: "Sample multiple CoT paths with temperature > 0, take the majority vote. More robust than a single CoT call because it averages out noise."
- Walk through the code: five samples, extract answer from each, majority vote.
- Structured prompting: "In production, free-form text is unparseable. You need to specify a schema — JSON, a table, typed fields."
- "Kaizen Signature enforces this at the framework level, so you do not have to parse LLM output with regex. We get to Signature on the next slide."
- "If beginners look confused": "Self-consistency is like asking the same question to a room full of experts and taking the majority answer. Each individual might be wrong sometimes, but the majority is usually right."
- "If experts look bored": "Self-consistency is strictly better than CoT on every reasoning benchmark, but it multiplies cost by N. The engineering tradeoff is when the stakes justify the spend." **Transition**: "Let us put all of these techniques on one slide."

Slide 13: Prompt Engineering — The Complete Toolkit

Time: ~2 min **Talking points**:

- Walk through the table once. Do not read every row.
- Emphasise the engineering rule at the bottom: "Start with zero-shot. Add few-shot if accuracy is insufficient. Add CoT if reasoning is needed. Add self-consistency only if the cost is justified by the stakes."
- "Self-consistency is expensive: it multiplies your API bill by N. Use it when you are making a high-stakes decision, not when you are classifying news articles."
- "If beginners look confused": "This is the decision tree. You never need all of them at once. Start simple, escalate when needed."
- "If experts look bored": "The accuracy gains in the table are median values from benchmarks. Your mileage will vary by task. The point is the ordering, not the exact numbers." **Transition**: "Now let us see how Kaizen wraps all of this in a type-safe interface."

Slide 14: Kaizen — Delegate & Signature

Time: ~3 min **Talking points**:

- Walk through the code. A Signature class defines the inputs and outputs as typed fields. The Delegate handles the LLM call.
- "This is the Kailash bridge: everything we just learned about prompting is wrapped in a type-safe interface. No more parsing JSON from raw text. The Delegate handles streaming, retries, and cost tracking."
- Show the cost attribute explicitly. "Students should always know how much they are spending. The Delegate makes it visible."
- "If the LLM fails to return the schema, Kaizen retries with corrective prompting. You do not need to handle that yourself."
- "If beginners look confused": "Think of Signature as a form. The LLM has to fill in the fields. If it does not, Kaizen asks again."
- "If experts look bored": "The Delegate supports streaming, structured decoding, function calling, and cost tracking out of the box. It is a production wrapper around the raw API." **Transition**: "A quick aside on inference optimisation, then we get to the exercise."

Slide 15: Inference Optimisation (Brief)

Time: ~2 min **Talking points**:

- Three optimisations everyone should know: KV-cache (caches previously computed tokens), speculative decoding (draft model proposes, large model verifies), continuous batching (requests join the batch as slots free up).
- "You will not implement these yourself. But understanding them explains why some APIs are faster than others, and why batch pricing exists."
- "Lesson 6.8 covers vLLM, which is the production serving framework that bundles all of these."
- "If beginners look confused": "Skip the details. The point is that calling an LLM API is not the same as the API running fast. There is a lot of engineering behind the scenes."
- "If experts look bored": "Flash Attention is the one missing from this slide; it shows up in 6.8. PagedAttention is the interesting vLLM contribution." **Transition**: "Time for the first exercise."

Slide 16: Exercise 6.1 — Prompt Engineering Showdown

Time: ~2 min **Talking points**:

- Walk through the exercise skeleton. Students implement zero_shot_classify, few_shot_classify, and cot_classify — three functions using the same Kaizen Signature.
- "Run them on the same test set. Measure accuracy and cost for each. The table at the end is the deliverable."
- "This is not a toy exercise. Cost tracking is the engineering differentiator — it is a production evaluation, not a demo."
- "If beginners look confused": "You only need to write three small functions. The Signature and Delegate do the heavy lifting."
- "If experts look bored": "Stretch task: add self-consistency as a fourth approach and compare the cost-accuracy tradeoff." **Transition**: "Lesson 6.2. Fine-tuning. This is the mathematical heart of the module."

Slide 17: Lesson 6.2 — LLM Fine-tuning, LoRA, Adapters & the Technique Landscape

Time: ~1 min **Talking points**:

- Read the learning objectives. Five outcomes, two from-scratch implementations.
- "This lesson is the most mathematically dense in the module. If you only follow the green foundations slides here, that is fine — you will still come out knowing what LoRA and adapters are and when to use each."
- Connect to M4: "Remember SVD from M4.3? LoRA is literally SVD applied to weight updates. If you followed that, you will follow this." **Transition**: "Let us start with LoRA theory."

Slide 18: LoRA — Low-Rank Adaptation

Time: ~4 min **Talking points**:

- Core idea: "Pre-trained weights W_0 are frozen. Instead of updating all d by d parameters, we learn two small matrices B (d by r) and A (r by d). Rank r is much smaller than d — typically 4, 8, or 16."
- Walk through the forward equation: h = W_0 x + BA x. "The original output plus a low-rank adaptation."
- Point at the parameter savings box: "For d=4096 and r=8, the full update is 16.7 million parameters. LoRA is 65 thousand. That is a 256x reduction."
- M4 connection: "LoRA IS SVD applied to weight updates. The low-rank matrices B and A approximate the full update delta-W just like truncated SVD approximates a matrix."
- "If beginners look confused": "Think of it as summarising a 1000-page book into a 4-page summary. You lose some detail but capture the essence. And you only need to store the 4-page summary, not the 1000 pages."
- "If experts look bored": "The interesting question is the rank selection. Ranks 4 to 16 are typical, but the optimal rank depends on the task. For some tasks, rank 1 is enough." **Transition**: "Now let us implement it from scratch."

Slide 19: LoRA — From-Scratch Implementation

Time: ~4 min **Talking points**:

- Walk through every line of the LoRALayer class.
- Key points: the original linear layer has requires_grad=False — it is frozen. Only A and B are trainable.
- Initialisation matters: A is initialised with small random values, B with zeros. "At initialisation, B times A equals zero, so the model starts as the original pre-trained model. Training gradually learns the adaptation. This is why LoRA is safe: you cannot make the model worse at the start."
- Forward pass: compute original + adaptation, add them.
- "If beginners look confused": "The original model does its thing. The LoRA layer adds a small correction on top. At the start, the correction is zero."
- "If experts look bored": "Notice how the adaptation is computed as (x @ A.T) @ B.T. That is the order that matters for efficiency — never materialise the full delta-W." **Transition**: "LoRA was not the first parameter-efficient method. Adapters came before."

Slide 20: Adapter Layers — Bottleneck Modules

Time: ~3 min **Talking points**:

- Architecture: down-project to a bottleneck, apply a non-linearity, up-project back. Residual connection around the whole thing.
- Historical note: "Adapters predate LoRA — Houlsby et al. 2019. Same core idea: freeze the big model, train small additions."
- The bottleneck forces the adapter to learn a compressed representation of the task-specific knowledge.
- Init trick: up-projection weights and bias are initialised to zero. "Just like LoRA's B=0 trick, this means the adapter output is zero at init. Training starts from the original behaviour."
- "If beginners look confused": "Think of adapters as tiny specialist modules plugged into the side of the big model."
- "If experts look bored": "Adapters are sequential — they add computation to every forward pass. That is their main disadvantage vs LoRA, which we cover on the next slide." **Transition**: "Head-to-head comparison."

Slide 21: LoRA vs Adapters — Head-to-Head

Time: ~2 min **Talking points**:

- Walk through the table quickly. The key differentiator is inference latency: LoRA can be merged into the base weights after training, so inference is exactly the same speed as the original model. Adapters add sequential computation at every forward pass.
- "In production, LoRA dominates because merged weights mean zero inference overhead. Adapters remain useful when you need to swap between many tasks without reloading the model."
- Modularity trade-off: adapters are separate modules you can hot-swap. LoRA can be merged (fast) or left separate (flexible).
- "If beginners look confused": "LoRA is the default. Use adapters only if you need to swap between many tasks at serving time."
- "If experts look bored": "The inference-latency argument is why LoRA won the production battle. But for multi-task serving with many small tasks, adapters can still win." **Transition**: "LoRA and adapters are just two of ten techniques. Let us look at the full landscape."

Slide 22: The Fine-tuning Landscape — 10 Techniques

Time: ~4 min **Talking points**:

- Do not deep-dive every technique. Walk through the table once.
- Highlights: Prefix Tuning (K/V prefix vectors), Prompt Tuning (learnable soft prompts), LLRD (lower learning rate for earlier layers — they are more general), Progressive Freezing (unfreeze top-down), Distillation (teacher-student), DP-SGD (gradient noise for privacy), EWC (Fisher Information to prevent catastrophic forgetting).
- Point at the decision rule: "Start with LoRA. Full fine-tuning only if the domain is very different. Distillation for constrained hardware. DP-SGD when privacy is a legal requirement."
- "If beginners look confused": "You do not need to memorise these. The point is: LoRA is almost always the right starting point. Only escalate when you have a specific reason."
- "If experts look bored": "EWC is the interesting one for continual learning scenarios. The Fisher Information penalty prevents the model from forgetting previous tasks when you add new ones." **Transition**: "Once you have multiple fine-tuned models, you can combine them."

Slide 23: Model Merging — Combining Fine-tuned Models

Time: ~3 min **Talking points**:

- Four techniques: TIES (trim, elect sign, merge), DARE (drop and rescale), SLERP (spherical linear interpolation), Task Arithmetic (add/subtract task vectors).
- Core insight: "Fine-tuned weights form 'task vectors' that can be added and subtracted like arithmetic. You can combine skills without retraining."
- Application: "Train separate LoRA adapters for sentiment analysis and summarisation. Merge them with TIES to get a model that does both — without retraining."
- "If beginners look confused": "Think of it as mixing paint colours. Each adapter is a colour; merging creates a new shade."
- "If experts look bored": "Task arithmetic is the most theoretically interesting: subtract a bad behaviour vector to remove a capability, or add a good behaviour vector to gain one. The linear algebra of fine-tuning." **Transition**: "Brief aside on running these big models on small hardware."

Slide 24: Quantisation — Running Large Models on Small Hardware

Time: ~2 min **Talking points**:

- Four methods: GPTQ, AWQ, GGUF, bitsandbytes. All reduce precision from 16-bit or 32-bit to 4-bit or 8-bit.
- QLoRA is the practical combination: "Quantise the base model to 4-bit NF4. Add LoRA adapters in full BF16. Train LoRA while the base stays quantised. Result: fine-tune a 65B model on a single 48GB GPU."
- Dettmers et al. (2023) showed QLoRA matches full 16-bit fine-tuning quality with 4x less memory.
- "If beginners look confused": "Quantisation is like compressing a photo. You lose a tiny bit of quality but the file is 4x smaller. QLoRA means you can fine-tune large models on consumer hardware."
- "If experts look bored": "The NF4 datatype is specifically designed for normally distributed weights. It quantises the outliers differently from the bulk. The math is elegant." **Transition**: "Now let us see how Kailash wraps all of this in one pipeline."

Slide 25: kailash-align — Fine-tuning with AlignmentPipeline

Time: ~2 min **Talking points**:

- Walk through the code. Create an AlignmentConfig with method="lora", set the rank, alpha, target modules, learning rate, and epochs. Then call pipeline.train with a dataset.
- "AdapterRegistry stores versioned adapters. You can register them by name and swap them in at deployment time."
- "The Kailash bridge: everything we learned about LoRA, adapters, and quantisation is wrapped in AlignmentPipeline. Students implement from scratch first, then see how the engine does it."
- "If beginners look confused": "You do not start with this. You start with the from-scratch implementation. AlignmentPipeline is what you use once you understand what it is doing."
- "If experts look bored": "Trainable params comes out around 262K for an 8B model with rank 8 on q_proj and v_proj. That is less than 0.01% of the parameters." **Transition**: "Time for the second exercise."

Slide 26: Exercise 6.2 — From-Scratch Fine-tuning

Time: ~2 min **Talking points**:

- Walk through the five tasks: implement LoRALayer from scratch, implement AdapterLayer from scratch, fine-tune both on IMDB, compare, merge two LoRA adapters with TIES.
- Flag the common mistake: forgetting to freeze the original weights. "If W_0.requires_grad is True, you are doing full fine-tuning, not LoRA. Check this."
- The merging task is a stretch goal for advanced students.
- "If beginners look confused": "The first two tasks are the important ones. Get LoRA and Adapter working on IMDB. Skip the merging if you run out of time."
- "If experts look bored": "The merging task is where it gets interesting. TIES will give you different results from naive averaging. Measure and explain." **Transition**: "Lesson 6.3. Preference alignment. This is where we stop teaching the model what to say and start teaching it what is good."

Slide 27: Lesson 6.3 — Preference Alignment: DPO & GRPO

Time: ~1 min **Talking points**:

- Read the four learning objectives.
- "DPO is the breakthrough: it removes the reward model from RLHF, making alignment accessible to anyone who can fine-tune. GRPO is the 2025 extension used in DeepSeek-R1 — even more efficient, even simpler."
- "This lesson is about making models not just capable, but aligned with human preferences." **Transition**: "Let us derive DPO."

Slide 28: DPO — Direct Preference Optimization

Time: ~5 min **Talking points**:

- The derivation sketch: "RLHF objective is maximise reward minus KL penalty to reference. Rafailov et al. (2023) showed the optimal policy has a closed-form solution. Rearranging, we can express the reward in terms of the policy directly. The reward model drops out of the equation entirely."
- Walk through the DPO loss term by term. Point at the log ratio pi/pi_ref: "This measures how much the policy has changed from the reference for each response."
- The sigma (sigmoid) converts the difference into a probability — the probability that the policy prefers the chosen response over the rejected one.

- Beta is the temperature: "Higher beta means stricter adherence to the reference. Lower beta means more freedom to change."
- Bradley-Terry model: "The assumption that preferences follow a logistic function of reward differences. This is where the sigmoid comes from."
- "If beginners look confused": "DPO says: given two answers, make the good one more likely and the bad one less likely, but do not change too much from the original model. That is it."
- "If experts look bored": "The key insight is that the reward model never existed in DPO — the policy IS the reward model, up to a constant. This is elegant and it is also why DPO is so much more stable than PPO." **Transition**: "Now the implementation."

Slide 29: DPO — Training Loop

Time: ~3 min **Talking points**:

- Walk through the code. The dpo_loss function is four lines of math. That is the entire algorithm.
- Key operational points: "You need two models in memory — the policy (which is trained) and the reference (which is frozen). You do NOT need a reward model."
- The ref model is typically the SFT checkpoint. "It anchors the training — the policy cannot wander too far."
- "In practice, you compute log probs by running both models on the chosen and rejected sequences, then plug into the loss."
- "If beginners look confused": "Four lines of math. No reward model. No PPO. No instability. That is why DPO spread so quickly."
- "If experts look bored": "The implementation is clean, but the memory cost is still 2x a single model. QLoRA fixes this by quantising the reference model." **Transition**: "DPO was 2023. Let us look at the 2025 evolution."

Slide 30: GRPO — Group Relative Policy Optimization

Time: ~4 min **Talking points**:

- GRPO is what powers DeepSeek-R1's reasoning ability. Walk through the algorithm: sample G completions per prompt, score each with a verifier (not a learned reward model), compute advantage relative to group mean, update policy with clipped objective.
- Walk through the advantage formula: "Reward minus group mean, divided by group standard deviation. This is group-normalised advantage."
- DPO vs GRPO table: "DPO uses preference pairs; GRPO uses single prompts with multiple completions. DPO is better for subjective quality; GRPO is better for objective tasks like code and math where you can verify correctness."
- "If beginners look confused": "Imagine giving 10 students the same maths problem, marking all their answers, then telling each student how they did compared to the group average. Students above average get positive feedback; students below get negative."
- "If experts look bored": "The verifier is the interesting part. For code, it is 'does it compile and pass the test suite'. For math, it is 'is the final answer correct'. For subjective tasks, you cannot use GRPO — that is when DPO shines." **Transition**: "Once you have a fine-tuned model, how do you evaluate it?"

Slide 31: LLM-as-Judge Evaluation

Time: ~3 min **Talking points**:

- The idea: use one LLM to rate another LLM's outputs. Walk through the judge prompt.
- Three known biases: position bias (whichever response is shown first gets a boost), verbosity bias (longer responses rated higher), self-enhancement (a model rates its own outputs higher).
- Mitigations are mechanical: swap positions and average, normalise by length, use a different model as judge.
- "Never trust a single judge call. Always swap positions, run multiple times, aggregate. LLM-as-judge is a statistical estimator, not a deterministic oracle."
- "If beginners look confused": "It is like asking a human to rate two essays. If you always show essay A first, essay A gets a slight unfair advantage. So you flip the order and average."
- "If experts look bored": "MT-Bench uses GPT-4 as judge. The self-enhancement bias is real and measurable — GPT-4 rates GPT-4 outputs about 10% higher than equivalent Claude outputs." **Transition**: "LLM-as-judge is fast but biased. For standard benchmarks, we use established evaluation sets."

Slide 32: Evaluation Benchmarks

Time: ~2 min **Talking points**:

- Quick tour: MMLU (multi-task language understanding, 57 subjects), HellaSwag (commonsense), HumanEval (code), MT-Bench (LLM-as-judge multi-turn), GSM8K (grade-school math).
- lm-eval-harness is the unified framework. One command, many benchmarks.
- Critical concept: alignment tax. "Run benchmarks before and after alignment. If MMLU drops more than 2%, your alignment is degrading general capability. You have paid too much for the alignment you gained."
- "If beginners look confused": "Benchmarks are standardised tests for LLMs. They tell you what capabilities the model has."
- "If experts look bored": "lm-eval-harness supports 200+ benchmarks. The interesting question is benchmark contamination — when the test set is in the training data." **Transition**: "Let us see how Kailash wraps DPO in one call."

Slide 33: kailash-align — DPO Training

Time: ~2 min **Talking points**:

- Walk through the code. AlignmentConfig with method="dpo", beta=0.1, LoRA rank for efficiency. pipeline.train on preference pairs, eval_with="llm_judge".
- "Students implement the loss function from scratch first, then use the pipeline. The pipeline is the shortcut; the from-scratch is the understanding."
- Output includes win_rate vs reference and mmlu_delta — the alignment tax tracker.
- "If beginners look confused": "You are not writing DPO from scratch in production. You write it once to understand, then you use the pipeline."
- "If experts look bored": "The interesting production pattern is evaluating win_rate and mmlu_delta together. You want win_rate up and mmlu_delta near zero. If mmlu_delta is -5%, you have alignment-tax'd your way to a dumber model." **Transition**: "Time for the third exercise."

Slide 34: Exercise 6.3 — Preference Alignment

Time: ~1 min **Talking points**:

- Four tasks: fine-tune with DPO, evaluate with LLM-as-judge measuring position and verbosity bias, run lm-eval benchmarks before and after, report the alignment tax.
- "Students must demonstrate two things: alignment worked (win rate up) AND capability was preserved (benchmarks stable). Both are required."
- "If beginners look confused": "The hardest part is measuring the biases. Start with position bias: run each evaluation twice, once with each order, and average."
- "If experts look bored": "Stretch: implement the bias correction as a post-hoc adjustment. You can quantify and remove position bias from any LLM-as-judge system." **Transition**: "Lesson 6.4. RAG. The most-deployed LLM pattern in production."

Slide 35: Lesson 6.4 — RAG Systems

Time: ~1 min **Talking points**:

- Read the four learning objectives.
- "RAG is the most deployed LLM pattern in production. It solves the biggest LLM limitation: hallucination. By grounding the model in retrieved documents, we get factual, verifiable answers."
- Prerequisites: 6.1 prompting plus M4 Lesson 6 embeddings. **Transition**: "Let us start with the concept."

Slide 36: RAG — Retrieval-Augmented Generation

Time: ~3 min **Talking points**:

- Walk through the pipeline diagram. Offline: chunk documents, embed, index. Online: embed query, retrieve similar chunks, generate with query + chunks.
- Why RAG beats fine-tuning for knowledge: "LLMs have a knowledge cutoff. LLMs hallucinate. RAG grounds the model in actual documents. And updating documents is much cheaper than retraining weights."
- "RAG is an open-book exam for the LLM: it can look up the answer instead of relying on memory."
- "If beginners look confused": "Think of RAG as giving the AI a reference book. Instead of guessing, it looks up the answer. And you can update the reference book without retraining the AI."
- "If experts look bored": "The interesting question is when to RAG vs when to fine-tune. Rule of thumb: RAG for facts, fine-tune for style. RAG for updatable knowledge, fine-tune for fixed capabilities." **Transition**: "The most underrated decision in RAG: chunking."

Slide 37: Chunking Strategies

Time: ~3 min **Talking points**:

- Four strategies: fixed-size (simple, breaks mid-sentence), sentence (coherent but variable), paragraph (preserves context but can be large), semantic (expensive but meaning-preserving).
- Overlap: 10-20% overlap between chunks. "Without overlap, a fact split across two chunks is lost to both."
- Chunk size: 256-512 tokens is the sweet spot for most use cases. Smaller means precise but noisy; larger means more context but less precise.
- "If beginners look confused": "Imagine cutting a textbook into note cards. Too small and each card is useless. Too big and you cannot find what you need. Around 300-400 words per card is usually right."
- "If experts look bored": "Semantic chunking is the current frontier. Embed every sentence, cluster by similarity, use cluster boundaries as chunk boundaries. Slow but it produces the best retrieval." **Transition**: "Once you have chunks, how do you find the relevant ones?"

Slide 38: Retrieval — Dense, Sparse, and Hybrid

Time: ~4 min **Talking points**:

- Three methods:
  - Dense: embed query and documents, cosine similarity. Captures semantic meaning.
  - Sparse (BM25): term frequency + inverse document frequency. Exact keyword matching, fast, no GPU, interpretable.
  - Hybrid: reciprocal rank fusion combining both. Best of both worlds.
- "In practice, hybrid almost always wins. Dense captures 'what you mean', sparse captures 'what you say', and some queries need both."
- Walk through the RRF code briefly. "It is a one-liner: 1/(k+rank) summed across methods."
- "If beginners look confused": "Dense is like asking a librarian who understands your topic. Sparse is like searching the index at the back of the book. Hybrid is doing both and combining the results."
- "If experts look bored": "The reciprocal rank fusion k=60 is a magic number from the IR literature. It is surprisingly robust across domains." **Transition**: "Two advanced techniques: re-ranking and HyDE."

Slide 39: Re-ranking & HyDE

Time: ~3 min **Talking points**:

- Re-ranking: "Two stages. First stage is fast but approximate — bi-encoder retrieval. Second stage is slow but accurate — cross-encoder re-ranking. The cross-encoder sees query and document together, so it scores relevance much better."
- "This is the standard production pattern: retrieve 50 candidates fast, re-rank to top 3 accurately."
- HyDE: "Query and document are in different 'language spaces'. A question looks different from an answer. So we ask the LLM to generate a hypothetical answer, and we embed that. The hypothetical answer does not need to be correct — it just needs to be in the right neighbourhood."
- "If beginners look confused": "Re-ranking is like having a fast librarian grab a stack of candidates, then a slow expert look through the stack carefully. HyDE is more surprising — you ask the AI to guess an answer, then use the guess to find real answers."
- "If experts look bored": "HyDE works because embedding distances are shorter between answers than between questions and answers. The hypothetical answer bridges the gap." **Transition**: "How do you know if your RAG system is actually good?"

Slide 40: RAGAS — RAG Evaluation Framework

Time: ~3 min **Talking points**:

- Four metrics: Faithfulness (is the answer supported by the context?), Answer Relevance (does it address the question?), Context Relevance (are the retrieved chunks relevant?), Context Recall (did we retrieve everything needed?).
- Faithfulness is the most important: "If the answer is not grounded in the retrieved documents, you have a hallucinating RAG system. Faithfulness is the hallucination detector."
- Walk through the ragas.evaluate code. One call returns all four metrics.
- "If beginners look confused": "Faithfulness asks: did the AI make this up, or did it actually find it in the documents? That is the only question that matters for RAG."
- "If experts look bored": "The four metrics are two pairs: quality of retrieval (context relevance, context recall) and quality of generation given retrieval (faithfulness, answer relevance). Diagnose failures by

separating the two." **Transition**: "Time for the fourth exercise."

Slide 41: Exercise 6.4 — Build a RAG System

Time: ~1 min **Talking points**:

- Four tasks: build RAG on Singapore policy documents, compare BM25/dense/hybrid, evaluate with RAGAS, implement HyDE and measure improvement.
- "This is a full engineering exercise. Production-grade RAG pipeline with proper evaluation. The Singapore policy dataset is public-domain and ensures real-world complexity."
- "If beginners look confused": "Start with dense retrieval only. Get the pipeline end-to-end, then add BM25, then hybrid, then HyDE. Incremental is fine."
- "If experts look bored": "Stretch: experiment with chunking sizes on the same dataset. You will find that chunk size matters more than retrieval method." **Transition**: "Lesson 6.5. Agents. This is where LLMs become autonomous."

Slide 42: Lesson 6.5 — AI Agents: ReAct, Tool Use & Function Calling

Time: ~1 min **Talking points**:

- Read the four learning objectives.
- "We move from passive LLM usage (prompting, fine-tuning, RAG) to active LLM usage: agents that reason, take actions, and observe results. This is where LLMs become autonomous."
- Prerequisite: 6.1 Kaizen Delegate. "You already know how to call an LLM. Now you will let the LLM decide what to do next." **Transition**: "What exactly is an agent?"

Slide 43: What Is an AI Agent?

Time: ~3 min **Talking points**:

- Core loop: observe, think, act, observe again. Repeat until the task is done or the budget is gone.
- "An agent is an LLM that can use tools. Instead of just generating text, it can search the web, run code, query databases, call APIs."
- Pipeline vs agent table: pipelines are deterministic with fixed steps; agents are dynamic and decide their own next step. Pipelines are fast; agents are flexible.
- Engineering rule: "If a pipeline can solve it, do not use an agent. Agents add complexity and cost. Use agents when the problem requires reasoning about which tools to use."
- "If beginners look confused": "A pipeline is like a recipe. An agent is like a chef who decides what to cook based on what is in the fridge. Recipes are fast and predictable; chefs are flexible and creative."
- "If experts look bored": "The interesting design question: how much agency to grant. Too little and you might as well use a pipeline. Too much and the agent wanders. The right answer is usually less than you think." **Transition**: "The ReAct pattern is the foundational agent loop."

Slide 44: ReAct — Reasoning + Acting

Time: ~4 min **Talking points**:

- Walk through the example trace step by step. Thought, Action, Observation, repeat. Each thought plans the next action; each observation updates the plan.
- "Without 'Thought' steps, agents make random tool calls. The reasoning step forces the model to plan before acting, which dramatically improves tool selection."
- Yao et al. (2023): ReAct outperforms act-only agents on HotpotQA by 6%, reduces hallucination rate by 40%.
- "The 'Thought' step is not overhead; it is the mechanism. It is also the debugging artifact — you can read the trace and see why the agent did what it did."
- "If beginners look confused": "It is like thinking out loud before doing something. You plan, then act, then check the result. Humans do this all the time; ReAct makes the LLM do it too."
- "If experts look bored": "The interesting thing is that pure act-only agents hallucinate MORE than ReAct agents. Reasoning constrains the action space." **Transition**: "The mechanism for actually doing things is function calling."

Slide 45: Function Calling — Structured Tool Use

Time: ~3 min **Talking points**:

- JSON schema defines a tool: name, description, parameters with types and descriptions, required fields.
- Tool choice parameter: auto (model decides), required (must call something), specific (must call a named tool).
- Parallel function calling: "Models can invoke multiple tools simultaneously when the calls are independent. Reduces latency for multi-tool queries."
- "Function calling is the bridge between LLM reasoning and real-world actions. The LLM generates a structured JSON call; your code executes it and returns the result."
- "If beginners look confused": "Think of it as the AI filling out a form to request an action, and your code processes the form. The schema is the form template."
- "If experts look bored": "Parallel function calling is a significant latency win for multi-step agents. The tradeoff is you lose sequential context between tool calls." **Transition**: "How do you design a good agent in the first place?"

Slide 46: Agent Design — The Hiring Framework

Time: ~3 min **Talking points**:

- Four questions: What is our goal? What is our thought process? What specialist would we hire? What tools do they need?
- "The key insight: vague agents produce vague results. Specific agents produce specific results."
- "You would not hire a 'general analyst'. You would hire a 'financial fraud investigator with experience in transaction pattern analysis'. The more specific your agent's role, the better it performs."
- Design considerations: iterative refinement (add a critic agent), human-in-the-loop (pause for validation on high-stakes decisions), monitoring (track intermediate outputs), cost budget (prevent runaway spending).
- "If beginners look confused": "Before building an agent, pretend you are writing a job description. The more precise the description, the better the candidate."
- "If experts look bored": "The sharpest heuristic is the specialist specificity. If you cannot name the agent's role in five words, the role is not specific enough." **Transition**: "Let us see how Kaizen implements all of this."

Slide 47: Kaizen — Building Agents

Time: ~3 min **Talking points**:

- Walk through the DataAnalystAgent code. BaseAgent is the base class. The @tool decorator registers methods as tools the agent can call.
- The agent constructor takes a Delegate (for LLM calls) and a max_cost (the safety net). agent.run takes a natural language task.
- "The tools wrap Kailash engines. The agent can use DataExplorer and TrainingPipeline as tools. This is the bridge between the ML engines you already know and agent-driven orchestration."
- Cost cap: "The max_cost parameter is the engineering safety net. The agent stops when it hits the budget."
- "If beginners look confused": "You are wrapping existing Kailash engines as tools. You already know how to use DataExplorer. Now you let an agent decide when to call it."
- "If experts look bored": "BaseAgent implements the ReAct loop under the hood. @tool auto-generates the JSON schema from type hints and docstrings. That is the production ergonomic." **Transition**: "A word on why cost budgets are non-negotiable."

Slide 48: Cost Budgets — Preventing Runaway Spending

Time: ~2 min **Talking points**:

- The problem: "A confused agent can make hundreds of LLM calls before realising it is stuck. At $0.003 per call, 500 calls equals $1.50 per request. A production system with 10K daily users equals $15,000/day."
- LLMCostTracker: budget is set, agent stops gracefully when exhausted, returns partial results, logs the reason.
- "Non-negotiable in production: every agent must have a cost budget. No exceptions. An unbounded agent is a financial liability."
- "If beginners look confused": "It is like giving a contractor a budget. They can spend up to that amount, then they stop and report what they accomplished."
- "If experts look bored": "The interesting engineering is graceful degradation. A well-designed agent that hits its budget should return its best partial answer, not an error." **Transition**: "Time for the fifth exercise."

Slide 49: Exercise 6.5 — Build a Data Analysis Agent

Time: ~1 min **Talking points**:

- Four tasks: build ReAct agent wrapping DataExplorer/TrainingPipeline/ModelVisualizer as tools, autonomously explore and model, enforce cost budget, debug the reasoning chain.
- The reasoning chain is the key deliverable: "Students must demonstrate the agent thinks before it acts."
- "If beginners look confused": "Start with one tool. Get the agent to call it correctly. Then add the others."
- "If experts look bored": "Stretch: add a critic agent that reviews the data scientist agent's output and suggests improvements. That is iterative refinement." **Transition**: "Lesson 6.6. One agent is good. Multiple specialists coordinating is better."

Slide 50: Lesson 6.6 — Multi-Agent Orchestration & MCP

Time: ~1 min **Talking points**:

- Read the four learning objectives.

- "Single agents are powerful but limited. Real systems need multiple specialists working together: one for data, one for modelling, one for reporting. This lesson covers the coordination patterns."
- "MCP is the interoperability standard that makes all of this work across frameworks." **Transition**: "Four coordination patterns."

Slide 51: Multi-Agent Patterns

Time: ~3 min **Talking points**:

- Four patterns with concrete examples:
  - Supervisor-worker: "A project manager delegating to a data scientist and a report writer."
  - Sequential: "Data cleaning agent feeds feature engineering agent feeds model training agent."
  - Parallel: "Search agent and compute agent run at the same time, results aggregated."
  - Handoff: "Customer support bot transfers to a billing specialist when the topic changes."
- Decision rule: "Start with sequential (simplest). Add parallelism if sub-tasks are independent. Use supervisor-worker only when task decomposition is dynamic."
- "If beginners look confused": "These are organisation charts for AI teams. Sequential is an assembly line. Parallel is a research lab. Supervisor-worker is a management hierarchy. Handoff is a customer service transfer."
- "If experts look bored": "The interesting question is when coordination overhead exceeds the parallelisation benefit. For small tasks, sequential always wins." **Transition**: "Let us implement supervisor-worker in Kaizen."

Slide 52: Multi-Agent — Supervisor-Worker in Kaizen

Time: ~3 min **Talking points**:

- Walk through the code. Specialist agents (DataScientistAgent, FeatureEngineerAgent) each have their own tools. The SupervisorAgent has a delegate_to tool that invokes a named specialist.
- "The supervisor treats other agents as tools. It reasons about which specialist to call, passes the task, and processes the result. This is the same ReAct loop, but the 'tools' are other agents."
- "If beginners look confused": "Think of the supervisor as an agent whose only tools are 'hire the data scientist' and 'hire the feature engineer'."
- "If experts look bored": "Recursive pattern: a supervisor can also be a specialist in a higher-level team. Kaizen supports arbitrary depth." **Transition**: "For agents to remember things across sessions, they need memory."

Slide 53: Agent Memory

Time: ~2 min **Talking points**:

- Three memory types:
  - Short-term: current conversation context, in the LLM's context window, lost when the session ends.
  - Long-term: persistent knowledge across sessions, stored in a vector database, retrieved by semantic similarity.
  - Entity: structured knowledge about specific people/projects/datasets, key-value with entity extraction.
- Production pattern: "Short-term for the current task. Long-term for domain knowledge. Entity for user-specific context. Never rely on context window alone for production agents."

- "If beginners look confused": "Short-term is what you remember during a meeting. Long-term is what you write in your notes. Entity is your contacts list."
- "If experts look bored": "Entity memory is the underused one. Most production agents fail at 'who are you and what were we talking about last time' even though the fix is trivial." **Transition**: "Let us talk about the tool protocol that makes all of this interoperable."

Slide 54: MCP — Model Context Protocol

Time: ~3 min **Talking points**:

- What MCP is: "Standardised protocol for exposing tools to AI agents. Tool registration with JSON schemas. Transport via stdio or HTTP/SSE. Any agent from any framework can use your MCP server."
- Why MCP matters: "Without MCP, every agent framework has its own tool format. With MCP, one server, any client. Like REST APIs for humans, MCP is APIs for AI agents."
- Walk through the kailash_mcp code. MCPServer, @server.tool decorator, server.run with a transport.
- "If beginners look confused": "Think of MCP as a USB port for AI tools. Any device that speaks USB can plug in. Any agent that speaks MCP can use your tools."
- "If experts look bored": "The interesting design tension is stateful vs stateless tools. MCP supports both, but stateful tools complicate the server implementation considerably." **Transition**: "MCP is how agents talk to tools. A2A is how agents talk to each other."

Slide 55: A2A — Agent-to-Agent Communication

Time: ~2 min **Talking points**:

- A2A: agents exchange typed messages, not free-form text. Agent Cards describe capabilities. Task lifecycle: submit, working, input-required, completed.
- Security concerns:
  - Data leakage: agents must not share data beyond their authorisation.
  - Prompt injection: one agent's output becomes another's input — sanitise.
  - Escalation attacks: Agent A asks Agent B to do something Agent A is not allowed to do.
- "Lesson 6.7 solves all three with PACT governance."
- "If beginners look confused": "When agents talk, every message is a potential trust boundary. You cannot assume the agent on the other side will protect your data."
- "If experts look bored": "The interesting question is whether agent-to-agent trust should be peer-to-peer or mediated by a central governance layer. PACT takes the second approach." **Transition**: "Time for the sixth exercise."

Slide 56: Exercise 6.6 — Multi-Agent ML Pipeline

Time: ~1 min **Talking points**:

- Four tasks: build multi-agent pipeline (DataScientist to FeatureEngineer to ModelSelector to ReportWriter), build MCP server exposing ML tools, test cross-agent coordination, verify structured outputs between agents.
- "Each agent stays within its specialisation. If the DataScientist starts writing reports, you have built a monolith, not a multi-agent system."
- "If beginners look confused": "Start with sequential pipeline. Each agent has one job. Test that each one works alone before chaining."

- "If experts look bored": "Stretch: add a critic agent that reviews the ReportWriter's output and sends it back for revision if it fails quality checks." **Transition**: "Lesson 6.7. Governance. This is what separates toy projects from production."

Slide 57: Lesson 6.7 — AI Governance Engineering

Time: ~1 min **Talking points**:

- Read the five learning objectives.
- Design principle: "This is ENGINEERING. Students implement access controls, test them, and verify they work. No philosophical discussion of AI ethics. The code IS the governance."
- "Governance without tests is governance theatre. If you cannot write a test that proves your access control works, it does not work." **Transition**: "PACT starts with addressing: who is asking for what."

Slide 58: PACT — D/T/R Addressing

Time: ~3 min **Talking points**:

- D/T/R: Domain, Team, Role. Walk through the structure: finance/risk-analysis/analyst.
- "Every agent, every API call, every data access has an address. The GovernanceEngine checks the address against the rules."
- Walk through the code. compile_org defines the org structure. Address identifies the requester. can_access returns a boolean; explain_access returns a reason.
- "If beginners look confused": "D/T/R is like a postal address for permissions. The system checks your address to decide what you can access."
- "If experts look bored": "The interesting part is explain_access. Every governance system should be able to explain its decisions, not just render them." **Transition**: "Once you can identify the requester, you need rules about what they can do."

Slide 59: Operating Envelopes

Time: ~3 min **Talking points**:

- Operating envelope defines what an agent can do: allowed tasks, forbidden tasks, cost limits, enforcement mode (warn, block, audit).
- Monotonic tightening: "Child envelopes inherit parent restrictions and can only add more. They can never loosen. This prevents privilege escalation."
- Fail-closed: "If the governance check fails (error, timeout), access is denied. Never fail-open. A broken guard is not a free pass."
- "If beginners look confused": "An operating envelope is like a job description with hard limits. The agent can do anything within the envelope, but nothing outside it."
- "If experts look bored": "Monotonic tightening is the key property. Any child's envelope must be a subset of its parent's envelope. The lattice of envelopes forms a partial order." **Transition**: "Budgets are cost envelopes. They cascade."

Slide 60: Budget Cascading

Time: ~2 min **Talking points**:

- How it works: parent agent has a total budget, allocates portions to children, children cannot exceed their allocation, parent can reclaim and reallocate remaining budget dynamically.
- Walk through the BudgetAllocator code. allocate by name, reclaim when finished, reallocate to another agent.
- "Budget cascading prevents any single agent from consuming the entire budget. The dynamic reallocation pattern is important: if one agent finishes under budget, the surplus can be redirected to another that needs more."
- "If beginners look confused": "It is like a project manager splitting a budget across team members, then reallocating surplus from under-spenders to over-runners."
- "If experts look bored": "The interesting question is whether budgets should be hierarchical or flat. PACT uses hierarchical because it matches D/T/R structure." **Transition**: "Let us see the whole governance stack in one class."

Slide 61: PactGovernedAgent — Governance Built In

Time: ~3 min **Talking points**:

- Walk through the code. PactGovernedAgent combines: an address (D/T/R), a delegate (LLM), a governance engine (D/T/R rules), an envelope (what it can do).
- "The governance check happens before every action. Nothing gets past the envelope."
- Show the blocked action at the bottom: "Delete all customer records" raises a GovernanceError because the task is not in allowed_tasks.
- "If beginners look confused": "You wrap the agent. Every action the agent tries to take is checked against the rules first. If it is not allowed, it is blocked."
- "If experts look bored": "The interesting design is that governance is compositional. You can wrap any Kaizen agent in PactGovernedAgent — the governance is orthogonal to the agent logic." **Transition**: "And you must test this, or it does not exist."

Slide 62: Governance Testing — Proving Safety

Time: ~3 min **Talking points**:

- Testing imperative: "Governance without tests is governance theatre. Test that allowed actions succeed. Test that denied actions are blocked. Test that envelopes tighten correctly. Test that budget enforcement works at zero."
- Walk through the four test functions. Each one tests a different invariant.
- "Red team your own governance. If you cannot write a test that proves your access control works, it does not work."
- "If beginners look confused": "We write tests to prove the locks on the doors actually work. Pushing on a locked door is the test."
- "If experts look bored": "Property-based testing is the natural extension. Generate random envelopes and verify monotonic tightening as an invariant." **Transition**: "Governance also requires audit trails."

Slide 63: Audit Trails & Clearance Levels

Time: ~2 min **Talking points**:

- Audit trail: every access decision is logged with timestamp, requester, action, decision, reason. Immutable append-only log. Required for regulatory compliance, incident investigation, accountability.
- Walk through the code briefly. get_audit_trail returns entries; each entry has the decision and reason.
- Clearance levels: Public, Internal, Confidential, Restricted. "Clearance levels map to D/T/R roles. A finance/risk/analyst has Confidential clearance. A public/support/bot has Public clearance."
- "If beginners look confused": "An audit trail is like CCTV for your AI system. It records every decision for review. If something goes wrong, you can reconstruct exactly what happened."
- "If experts look bored": "The interesting compliance question is retention policy. How long do you keep the audit trail? That is a legal question, not a technical one." **Transition**: "Time for the seventh exercise."

Slide 64: Exercise 6.7 — Governed Multi-Agent System

Time: ~1 min **Talking points**:

- Five tasks: define D/T/R, set operating envelopes, implement budget cascading, write governance tests (allow AND deny), generate and inspect audit trail.
- "The governance tests are the most important deliverable. If the tests pass, the system is safe. If they do not exist, the system is not safe regardless of how clean the code looks."
- "If beginners look confused": "Start with two agents and two rules. Write a test that proves rule 1 allows something. Write a test that proves rule 2 blocks something. Expand from there."
- "If experts look bored": "Stretch: combine with 6.6's multi-agent pipeline. Every agent in the pipeline should have its own envelope." **Transition**: "And now, the capstone. Lesson 6.8. Time to ship."

Slide 65: Lesson 6.8 — Capstone: Full Production Platform

Time: ~1 min **Talking points**:

- Read the five learning objectives.
- "The capstone integrates everything. You are not building from scratch; you are connecting components you have already built in previous lessons. Emphasis is on deployment, monitoring, and production-readiness."
- Scaffolding: ~40%. "This is the integration exercise. The components exist. You assemble them." **Transition**: "The deployment layer is Nexus."

Slide 66: Nexus — One Codebase, Three Interfaces

Time: ~3 min **Talking points**:

- Walk through the Nexus code. Create an app, register a service (our governed agent), add middleware (auth, rate limiter), serve with api=True, cli=True, mcp=True.
- "Nexus is the deployment layer. You write your service once, and Nexus exposes it as a REST API (for web apps), a CLI (for developers), and an MCP server (for AI agents). No code duplication."
- "If beginners look confused": "Think of it as one restaurant kitchen that serves dine-in, takeaway, and delivery from the same menu. One kitchen, three interfaces."
- "If experts look bored": "The interesting architectural choice is that Nexus treats API, CLI, and MCP as equivalent channels. Most frameworks treat one as primary and the others as afterthoughts." **Transition**: "Nothing ships without auth."

Slide 67: Authentication & Authorisation

Time: ~3 min **Talking points**:

- RBAC: users have roles, roles have permissions, permissions map to Nexus endpoints. Walk through the @rbac decorator on routes.
- JWT: client obtains a token via /auth/token, then passes it in the Authorization header. Standard pattern.
- PACT D/T/R addresses map naturally to RBAC roles: "The same governance that controls agents controls human access."
- "Unauthenticated requests must be rejected with 401. No exceptions."
- "If beginners look confused": "RBAC is 'roles have permissions'. JWT is 'a token proves who you are'. You combine them: the token identifies the user, RBAC decides what the user's role can do."
- "If experts look bored": "The interesting unification is mapping human RBAC to agent D/T/R. One governance model, two kinds of actors." **Transition**: "Now let us see the whole stack end to end."

Slide 68: Full Platform Integration

Time: ~3 min **Talking points**:

- Walk through the flow: TrainingPipeline to DataFlow to Kaizen Agent to PACT Govern to Nexus Deploy to DriftMonitor.
- Walk through the table. Six packages, one pipeline. Every module you have completed contributes a piece.
- "This is the entire MLFP stack. M6 is where it all comes together. Every exercise from M1 onwards was preparation for this integration."
- Debugging traces: every agent action, every governance decision, every API call is traceable. "When something fails in production, you can reconstruct the entire chain."
- "If beginners look confused": "This is the assembly line. Each station does one thing, and the final product is a deployed AI system."
- "If experts look bored": "The interesting operational question is where failures cascade. A DriftMonitor alert triggers retraining; a PACT denial triggers an audit review; a Nexus 500 triggers a rollback. These are the SRE patterns." **Transition**: "Deployed models degrade. You must monitor."

Slide 69: Production Monitoring — DriftMonitor

Time: ~2 min **Talking points**:

- Why monitor: "Models degrade over time as data distributions shift. An accurate model today may be wrong tomorrow."
- Three types of drift: data drift (input distribution changes), concept drift (input-output relationship changes), performance drift (accuracy degrades).
- Walk through the DriftMonitor code. reference_data is the training distribution. check each batch. Drift detected triggers retraining.
- "DriftMonitor was introduced in M3. Here it is deployed in production. The key: monitoring is not optional. A deployed model without monitoring is a ticking time bomb."
- "If beginners look confused": "Think of it as a regular health check for your AI system. You take its temperature to catch problems early."
- "If experts look bored": "The interesting question is concept drift detection without labels. DriftMonitor uses statistical distribution tests on inputs, but confirming concept drift requires labelled feedback." **Transition**: "Agents need different debugging than models."

Slide 70: Debugging Agent Reasoning Chains

Time: ~2 min **Talking points**:

- Walk through the trace code. agent.run with trace=True, iterate through trace steps. Each step has a type (thought, action, observation, governance) and content.
- Common debugging patterns table: loop forever (ambiguous goal, missing tool), wrong tool (descriptions too similar), governance blocked (missing permission), budget exhausted (too many retries), incoherent reasoning (context overflow).
- "Debugging agents is different from debugging code. You read the reasoning trace, not a stack trace. The most common issue: the agent loops because it does not have the right tool. Fix the tools, not the prompts."
- "If beginners look confused": "The trace is the debug log. You read it in English. If the agent is confused, the trace shows you exactly where the confusion started."
- "If experts look bored": "The interesting observation is that most agent failures are tool design failures, not LLM failures. Fix the tools, and the agent usually improves dramatically." **Transition**: "Agents also need automated tests."

Slide 71: Testing Agentic Systems

Time: ~2 min **Talking points**:

- What to test: tool correctness (known input, known output), reasoning quality (correct tool selected), governance (blocked stays blocked), budget (stops at limit), end-to-end (correct final result).
- Walk through the test code. test_tool_selection inspects the trace for the expected tool name. test_end_to_end checks the output contains expected terms and the cost is under budget.
- "Agent testing is non-deterministic. You cannot test exact outputs, but you can test tool selection, governance enforcement, and budget compliance."
- "If beginners look confused": "You do not test what the agent says. You test what it does. Did it call the right tool? Did it stay within budget? Did governance block the bad actions?"
- "If experts look bored": "Property-based testing for agents is the frontier. Random prompts, check invariants (budget respected, no prohibited tool calls). The challenge is defining good properties." **Transition**: "Quick note on production inference optimisation."

Slide 72: Inference Optimisation — Production Serving

Time: ~2 min **Talking points**:

- vLLM: PagedAttention for efficient KV-cache memory, continuous batching for GPU utilisation, tensor parallelism for multi-GPU, OpenAI-compatible API.
- Flash Attention: tiling-based attention, reduces memory from O(n^2) to O(n), 2-4x faster than standard attention, built into most modern frameworks.
- "You do not implement these. You configure them. The engineering skill is knowing which optimisation applies to your deployment constraint — latency, throughput, or memory."
- "If beginners look confused": "Skip the details. The point is that serving LLMs in production needs special frameworks. vLLM is the standard."
- "If experts look bored": "PagedAttention is the interesting contribution. Treating KV-cache memory like OS page tables was a significant insight." **Transition**: "One more brief awareness slide before the capstone exercise."

Slide 73: Multimodal LLMs — Brief Awareness

Time: ~2 min **Talking points**:

- Vision-language models: GPT-4V (text + image), Gemini (native multimodal), LLaVA (open source).
- Applications: document understanding (OCR + reasoning), chart interpretation, visual question answering, multimodal RAG.
- "Trajectory: LLMs are becoming multimodal by default. The text-only era is ending. Future agents will see, hear, and read simultaneously. The same Kaizen patterns — Delegate, BaseAgent, tools — apply to multimodal models."
- "For this module: awareness only. The engineering patterns you have learned — prompting, agents, governance — transfer directly. The tools change; the architecture does not."
- "If beginners look confused": "Do not worry about this slide. It is a pointer for future learning, not something you need today."
- "If experts look bored": "The interesting question is whether multimodal agents need new governance primitives. Image inputs introduce new prompt injection vectors." **Transition**: "Time for the capstone."

Slide 74: Exercise 6.8 — Capstone: Deploy a Governed AI Platform

Time: ~2 min **Talking points**:

- Five tasks: deploy the M6 multi-agent system via Nexus, add JWT + RBAC, integrate DriftMonitor, test via API/CLI/MCP, verify governance at deployment level.
- "Students connect everything they built in 6.1-6.7 into a single deployed system. The emphasis is on integration, not new concepts."
- Criteria recap: three channels accessible, auth works (401 for unauth), drift monitoring active, governance enforced, complete audit trail.
- "If it deploys, authenticates, monitors, and governs correctly, you have passed. That is the entire bar for the capstone."
- "If beginners look confused": "Start with the Nexus deployment. Get it running without auth. Then add auth. Then add DriftMonitor. Incremental."
- "If experts look bored": "Stretch: run a real drift injection test. Deploy, send normal traffic, then send shifted traffic, verify DriftMonitor catches it and triggers retraining." **Transition**: "Let us step back and review."

[PAUSE FOR QUESTIONS — 3 min]

Slide 75: Key Formula Recap — Attention Mechanism

Time: ~2 min **Talking points**:

- Recap the attention formula from M5. "It is the foundation of everything in M6."
- Where it shows up: 6.1 (LLMs use multi-head attention), 6.2 (LoRA targets attention projection matrices), 6.4 (dense retrieval uses attention-based embeddings), 6.8 (Flash Attention optimises the computation).
- The sqrt(d_k) scaling: "Prevents dot products from growing too large, which would push softmax into saturated regions with near-zero gradients. This is why training is stable at scale."
- "If beginners look confused": "This formula is the engine inside every LLM. Everything we built in M6 runs on this. You do not need to derive it, you need to know it is there."
- "If experts look bored": "The interesting operational consequence of sqrt(d_k) scaling is that attention is still the dominant cost for long sequences. That is why Flash Attention matters." **Transition**: "All four

formulas in one place."

Slide 76: M6 Formula Summary

Time: ~2 min **Talking points**:

- Walk through the four formulas: LoRA (W = W_0 + BA), DPO loss, GRPO advantage, Attention.
- "Four formulas, four concepts: LoRA decomposes weight updates, DPO aligns with preferences, GRPO normalises rewards within a group, Attention is the computation substrate for all of them."
- "You do not need to memorise these. You need to know what problem each one solves."
- "If beginners look confused": "Each formula solves one problem. LoRA makes fine-tuning cheap. DPO makes alignment simple. GRPO makes verifiable-task training stable. Attention is the core LLM operation."
- "If experts look bored": "Write them down without looking. You should be able to. If not, revisit the corresponding lesson tonight." **Transition**: "The complete Kailash stack, mapped."

Slide 77: The Complete Kailash Stack

Time: ~2 min **Talking points**:

- Walk through the flow diagram once: kailash-ml, kailash-dataflow, kailash-kaizen, kailash-align, kailash-pact, kailash-nexus.
- Walk through the API reference table. This is the bookmark slide for the capstone.
- "Every Kailash API used in M6 is on this slide. Bookmark it for the capstone exercise."
- "If beginners look confused": "Screenshot this slide. When you are building the capstone and you cannot remember which class handles governance, this is your reference."
- "If experts look bored": "Note that kailash-mcp is the thinnest package of the six. It is a protocol implementation, not a business logic layer." **Transition**: "The journey, one last time."

Slide 78: The M6 Journey

Time: ~2 min **Talking points**:

- Walk through the timeline once: 6.1 to 6.8, one node each.
- The six-verb arc: Understand (6.1), Customise (6.2-6.3), Ground (6.4), Empower (6.5-6.6), Govern (6.7), Ship (6.8).
- "Every concept produces runnable code. There is no theory-only lesson in M6. If you completed the exercises, you have built a production-ready AI platform."
- "If beginners look confused": "The journey words — understand, customise, ground, empower, govern, ship — are your mental index. Each one points to 1-2 lessons."
- "If experts look bored": "Notice how the verbs compound. You cannot ship without governing. You cannot govern without empowering. Each step requires the previous." **Transition**: "What makes this module different from the average LLM course."

Slide 79: What Makes This Module Different

Time: ~2 min **Talking points**:

- Comparison grid: typical LLM course is “call the API, build a chatbot”. MLFP M6 is “implement LoRA from scratch, derive DPO, build governed multi-agent systems, deploy with monitoring”.
- “The difference: we teach you to build systems that are safe, tested, governed, and deployed. Not just systems that work in a notebook.”
- “If the module felt hard today, hard means you learned something real.”
- “If beginners look confused”: “It is okay if the math was hard. The engineering patterns — agents, governance, deployment — will still serve you even if the derivations fade.”
- “If experts look bored”: “This slide is for the rest of the room. Let it sit.” **Transition**: “Your decision framework for real work.”

Slide 80: Decision Framework — When to Use What

Time: ~2 min **Talking points**:

- Walk through the table. Problem on the left, solution on the right, lesson number for reference.
- “This is your cheat sheet. When you face a real problem, find it in the left column and the solution is in the right.”
- “This slide plus Slide 77 is the entire M6 reference card.”
- “If beginners look confused”: “You do not need to memorise this. Screenshot it. Refer back when you face a real problem.”
- “If experts look bored”: “The interesting decisions are the ambiguous ones. ‘Model needs domain knowledge’ could be RAG or fine-tuning. Use RAG first; fine-tune only when RAG is insufficient.” **Transition**: “And the mistakes to avoid.”

Slide 81: Common Mistakes to Avoid

Time: ~2 min **Talking points**:

- Walk through each mistake. The most common: fine-tuning when RAG would work (cheaper, faster, updatable). The most dangerous: no governance on agents (security liability).
- “These are the mistakes that every team makes once. Learn from ours.”
- Other entries: agent without cost budget (runaway spending), testing LLM exact output (tests flake), deploying without monitoring (silent degradation), agents for simple tasks (unnecessary cost).
- “If beginners look confused”: “If you only remember one line from this slide: every agent needs a cost budget. No exceptions.”
- “If experts look bored”: “The ‘testing LLM exact output’ one catches more senior engineers than juniors. Juniors know the output is non-deterministic. Seniors think they can work around it.” **Transition**: “Where M6 sits in the MLFP programme.”

Slide 82: M6 in the MLFP Curriculum

Time: ~2 min **Talking points**:

- Walk through what M6 builds on (M1 Python and Polars, M2 statistics, M3 supervised ML, M4 NLP and embeddings, M5 DL and transformers).
- Walk through what M6 delivers: complete LLM engineering skillset, from-scratch implementations, production deployment with governance, full Kailash stack integrated end-to-end.
- “M6 is the culmination of the MLFP programme. Every module contributed a building block. M6 assembles them into a production AI platform.”

- "If beginners look confused": "Every prerequisite you worried about in earlier modules has paid off today. M1 to M5 were not gatekeeping. They were preparation."
- "If experts look bored": "The interesting observation is that M6 consumes every package in the Kailash stack. No other module uses all six." **Transition**: "The spectrum view."

Slide 83: The ML Spectrum — Where LLMs Fit

Time: ~1 min **Talking points**:

- Walk through the module spectrum briefly. M1-M2 data and statistics, M3 supervised, M4 pattern discovery and language, M5 representation learning, M6 reasoning, agency, governance.
- Walk through the M6 lesson spectrum. Each lesson occupies a different position.
- "M6 sits at the top of the spectrum: from data understanding through to autonomous reasoning systems."
- "If beginners look confused": "This is the map. M6 is the far right, the most autonomous. Earlier modules are more constrained and more predictable."
- "If experts look bored": "The interesting trend is the governance column. It only appears in 6.7 but it is the thread that makes everything else deployable." **Transition**: "Check yourselves."

Slide 84: Self-Assessment — Can You…

Time: ~2 min **Talking points**:

- Ask students to mentally check off each item silently.
- Foundations (everyone): pre-training explanation, 5+ prompting techniques, Kaizen Delegate, RAG pipeline, ReAct agent, cost budgets, PACT access controls, Nexus deployment.
- Theory (stretch): implement LoRA from scratch, derive DPO, GRPO vs DPO, RAGAS metrics, MCP server, governance tests.
- Advanced (expert): model merging, QLoRA and quantisation, vLLM optimisations.
- "If you cannot check off all Foundations items, revisit the relevant exercises. If you can check off Theory items, you are ready for advanced work."
- "If beginners look confused": "If the Theory column feels out of reach, that is fine. Foundations is the bar for passing this module."
- "If experts look bored": "Everyone in the Advanced column should aim for all three items. Model merging is the one most commonly missed." **Transition**: "How you will be assessed."

Slide 85: End-of-Module Assessment

Time: ~2 min **Talking points**:

- Capstone project: deploy a complete governed AI system. Must include prompting, fine-tuning, RAG, agents, governance, deployment. 15-minute presentation plus 5 minutes Q&A. Live demo via API, CLI, and MCP.
- Comprehensive quiz: covers all 8 lessons. Mix of conceptual, code interpretation, formula application. Open-book (you may reference your exercises). 60 minutes.
- "Both are open-book because the skill is engineering, not memorisation. Can you build it, deploy it, govern it, and explain why your design decisions are correct? The code IS the answer."
- "If beginners look confused": "Open-book means you can look things up. What you cannot do is look up the thinking — you still need to know which tool applies to which problem."

- "If experts look bored": "The capstone grade weights governance and monitoring heavily. A deployed system without governance fails, regardless of how impressive the model is." **Transition**: "Resources for deeper study."

Slide 86: Resources & References

Time: ~1 min **Talking points**:

- Papers column: Hu et al. (LoRA), Houlsby et al. (Adapters), Rafailov et al. (DPO), Shao et al. (DeepSeek-R1/GRPO), Yao et al. (ReAct), Wei et al. (CoT), Kojima et al. (zero-shot CoT), Lewis et al. (RAG).
- Kailash documentation: kaizen, align, pact, nexus, mcp.
- Tools: lm-eval-harness, RAGAS, vLLM.
- "Reference slide. Papers are for depth. Kailash docs are your primary reference for the exercises."
- "If beginners look confused": "You do not need to read the papers. They are here if you want depth."
- "If experts look bored": "The DPO paper (Rafailov 2023) is the most elegant read on this list. Skim it tonight if you have energy." **Transition**: "The eight sentences we want you to remember."

Slide 87: Key Takeaways

Time: ~2 min **Talking points**:

- Read each takeaway aloud:
    1. Prompting is an engineering skill. Start simple, escalate by measured need.
    2. LoRA democratised fine-tuning. 256x fewer parameters, same quality.
    3. DPO eliminated the reward model. Alignment is now accessible.
    4. RAG is the production default. Ground LLMs in facts, not memory.
    5. Agents need tools, not just prompts. ReAct equals reasoning plus action.
    6. Multi-agent systems need coordination protocols. MCP for tools, A2A for agents.
    7. Governance is engineering, not philosophy. Code it. Test it. Ship it.
    8. Deploy with monitoring. A model without DriftMonitor is a liability.
- "If you remember nothing else from today, these eight sentences will serve you well."
- "If beginners look confused": "Write them down. Each one is a complete thought. You will hit every single one of these in real projects."
- "If experts look bored": "The takeaway you are most likely to forget under pressure is number 7. It is the one that quietly distinguishes production-grade systems from demos." **Transition**: "Pacing notes for next time — this is the instructor slide."

Slide 88: For the Instructor — Pacing Notes

Time: ~1 min (skip or briefly acknowledge for student audiences) **Talking points**:

- Instructor-only slide. If students are in the room, acknowledge briefly and move on. If this is an instructor-training session, walk through the table.
- Per-lesson estimates: 6.1 (2.5h), 6.2 (3.5h), 6.3 (2.5h), 6.4 (3h), 6.5 (3h), 6.6 (2.5h), 6.7 (2.5h), 6.8 (4h). Total ~23.5 hours of instructional content, compressed to 180 minutes for this overview delivery.
- Biggest risks: 6.2 (from-scratch implementations need debugging) and 6.8 (integration issues). Allocate buffer time for both.
- "If running behind: the fine-tuning landscape survey (slide 22) can be assigned as reading."
- "If beginners look confused": "This slide is not for you. Skip it."

- "If experts look bored": "Calibrate your next delivery against these estimates. Adjust as needed." **Transition**: "Let us take the final discussion."

Slide 89: Discussion Questions

Time: ~8-10 min **Talking points**:

- Open discussion. Let students discuss in pairs first (3-4 min), then share with the class (5-6 min).
- Reflect questions:
  1. RAG vs fine-tuning — give a concrete example where each is the right choice.
  2. What happens if you deploy an agent without governance? Worst case?
  3. Why does DPO not need a reward model while RLHF does?
- Apply questions:
  4. 10,000 internal documents — how would you build a policy Q&A system?
  5. AI customer support assistant — which Kailash packages and why?
- "These questions test application, not recall. There are no single correct answers. The quality of reasoning matters."
- "If beginners look confused": "Focus on questions 1 and 4. They are the most grounded in concrete scenarios."
- "If experts look bored": "Question 2 is the most interesting red-team exercise. Walk through the attack surface." **Transition**: "And we close."

Slide 90: Module 6 Complete

Time: ~2 min **Talking points**:

- Read the closing slide title and subtitle aloud: "Module 6 Complete. You can now build, fine-tune, align, ground, govern, and deploy production AI systems."
- Read the provocation: "The goal is not to use AI. The goal is to deploy AI that is safe, tested, governed, and useful."
- Thank the class. This is the final module of MLFP.
- "You have completed the entire MLFP programme. Six modules. From zero Python and polars (M1) to a deployed governed AI platform (M6). Everything connects. Everything you built along the way is a component of what you just shipped today."
- Remind them of the capstone deadline and the quiz schedule.
- Close with the broader point: "You came in asking how ML works. You leave knowing how to ship it safely. That is the difference between ML curiosity and ML engineering."
- "If beginners look confused": "You do not need to feel expert on day one. You need to feel capable. If you can build the capstone, you are capable."
- "If experts look bored": "The hardest part of this journey for strong practitioners is not the math — it is the discipline of governance and testing. That is what separates published models from deployed systems." **Transition**: "Congratulations on completing MLFP. Now go build something that matters."

[CLOSE — applause, photos, informal Q&A]

Instructor Notes

Pacing Summary

The overview delivery targets 180 minutes across 90 slides — an average of 2 minutes per slide. Lesson title slides and closing slides are shorter (~1 minute). Key formula slides (LoRA, DPO, GRPO) and exercise introductions run longer (~3-5 minutes). Build in two 1-minute pauses for questions (after Slide 5 and after Slide 74) and one 3-minute break if the room needs it.

Full classroom delivery of every exercise — as outlined on Slide 88 — takes approximately 23.5 hours across the 8 lessons. This speaker-notes document supports the compressed 180-minute overview that introduces every slide at survey depth.

Audience Calibration

The room will contain:

- **Novices** who completed M1-M5 and are building their first LLM application. Prioritise green FOUNDATIONS callouts and plain-language fallbacks ("If beginners look confused").
- **LLM practitioners** who have used ChatGPT APIs but never implemented LoRA or DPO. Prioritise blue THEORY callouts and derivation deep-dives.
- **ML researchers** who know the papers but have not built governed production systems. Prioritise purple ADVANCED callouts and governance/testing emphasis.

Every slide includes a fallback for both ends. Use them as needed; you do not need to read every bullet.

Critical Messages (Repeat Often)

1. Every agent needs a cost budget. No exceptions.
2. Governance without tests is governance theatre.
3. RAG first, fine-tune second.
4. The reasoning trace is the debug log.
5. Deploy with monitoring or do not deploy.

These five messages anchor the module. Reinforce them in every discussion.

Final Note

This is the final module of MLFP. When you close on Slide 90, you are closing the entire programme, not just one module. Honour the arc. Congratulate the class. Leave them with the message that they are now capable of shipping real AI systems, and that the engineering discipline they learned here — not the specific models or APIs — is what will remain valuable as the field evolves.